

MetricLens

Master PRD

Movement Mode + LiftMap Mode

Open-Source Python Library for Metric Decomposition and Opportunity Detection

v0.1.0 Build-Ready · v1.0 Expansion · v2.0 LiftMap · May 2026

Solo-built · Open-source · Parked Side Project

Movement Mode · **LiftMap Mode**

Version: Master PRD · May 2026

Package releases: v0.1.0 (Movement Mode, build-ready) · v1.0 (planned) · v2.0 (LiftMap, planned)

Build Archetype: Solo-built, open-source, parked side project

Honest Claim: MetricLens is a pip-installable Python library with two planned modes. Movement Mode decomposes metric changes into segment contributions, mix effects, and rate effects. LiftMap Mode ranks segments by untapped opportunity volume. Neither mode claims causality.

Target Signal: GitHub credibility · Blog authority · DS/Analytics interview positioning

Product Vision

MetricLens is built around two complementary analytical questions that arise in every data-driven organisation.

Movement Mode answers: *what changed, where did it change, and was it driven by mix shift, rate shift, or both?* A metric moved — revenue dropped 14%, conversion rate fell 2.8pp — and the analyst needs a quantitative decomposition that tells them which segments contributed most, whether the change was driven by changes in segment composition or in per-segment performance, and where to look first. Movement Mode provides this in a single library call with deterministic, structured JSON/Markdown/HTML output.

LiftMap Mode answers: *where is the upside?* Given a metric and a benchmark rate, LiftMap ranks segments by opportunity signal — the product of denominator volume and the gap between the benchmark rate and the segment's actual rate. High-volume segments that underperform the benchmark are the highest-opportunity targets for investigation or experiment design. LiftMap Mode is the complement to Movement Mode: Movement Mode says what went wrong; LiftMap says where to focus improvement effort.

Both modes share the same infrastructure: MetricSpec, SegmentAnalyzer, DataProfiler, and ReportBuilder. Both produce the same three output formats: JSON, Markdown, HTML. Both carry the same honest interpretation language throughout — signals not conclusions, contributions not causes, suggestions for investigation not automatic decisions.

Release plan: v0.1.0 ships Movement Mode only — build-ready, fully specified, no deferred stubs. v1 expands Movement Mode with CLI, Shapley attribution across dimensions, bootstrap confidence intervals, and optional LLM narrator. v2 adds LiftMap Mode as a second analysis mode on the same MetricLens object.

Document History

P1 — Demo data grain fixed. Changed demo dataset from order-level rows to daily segment-level aggregated rows (date × channel × device × city × category). Updated all demo analyses: AOV uses RatioMetric("revenue", "orders"), not AverageMetric.

P2 — Disappeared segment cross-term corrected. Under zero-fill convention, $\text{cross_term} = (0 - w_b) \times (0 - r_b) = +w_b \times r_b$ (positive, not zero). Identity holds.

P3 — Contribution % sign semantics clarified. Signed contributions sum to +100% when $\text{total_delta} > 0$ and -100% when $\text{total_delta} < 0$. Absolute sum can exceed 100% when segments offset.

P4 — Zero total delta handling added. When $|\text{total_delta}| < 1e-9$, $\text{direction} = \text{"FLAT"}$, $\text{contribution_pct} = \text{None}$ for all segments.

P5 — Zero baseline value handling added. When $\text{baseline_value} == 0$, $\text{relative_delta_pct} = \text{None}$.

P6 — Schema field names corrected. `rate_delta_pp` → `rate_delta`, `weight_delta_pp` → `weight_delta`. `display_unit` property added to `MetricSpec`.

P7 — Null dimension fill mechanism made explicit. Internal working copy only; original `DataFrame` never modified.

P8 — LLM extras removed from v0 pyproject. v0 ships dev extras only.

P9 — Version convention unified. PRD v0.1 · Package 0.1.0 · `schema_version` "0.1" · Changelog v0.1.0.

P10 — LiftMap future paragraph added. One paragraph in roadmap only — no schema, stubs, or code.

P11 — Explicit non-additions confirmed. LiftMap implementation, confidence field, AI-powered language all explicitly excluded from v0.

P12 — Roadmap milestone renamed. "v0.1 (4–6 weeks)" → "v0.2 — 4–6 weeks after v0.1.0 release".

P13 — IPL removed from roadmap. Replaced with "Optional second demo using SaaS/product analytics aggregated data to demonstrate domain generality."

P14 — Upgraded to Master PRD. Part I (Movement Mode v0.1) preserved intact. Part II (Movement Mode v1 Expansion) and Part III (LiftMap Mode v2) added as fully specified planned modules.

PART I — MOVEMENT MODE (v0.1.0 Build-Ready)

1. Executive Summary

MetricLens is an open-source Python library for metric movement decomposition. When a business metric changes between two time periods, MetricLens answers four questions: what changed, where did it change, which segments contributed most, and whether the movement was driven by volume shift, mix shift, or rate shift. It does not claim to explain why the metric changed.

The library is DataFrame-native, deterministic, pip-installable, and honest about its scope. It supports four metric types — SumMetric, CountMetric, RatioMetric, and AverageMetric — with different decomposition logic for additive metrics versus ratio/average metrics. The mathematical core is the three-way mix/rate/cross decomposition for ratio metrics, an exact identity that MetricLens reports fully, including the cross-term that most manual analyses discard.

v0 ships: four metric types, a data quality profiler, a delta engine, a per-dimension segment analyzer, a mix-shift decomposition engine for ratio/average metrics, and a report builder producing JSON, Markdown, and HTML output. No LLM. No agent. No CLI. No Shapley. No Streamlit. A complete pytest suite and a synthetic e-commerce demo using daily aggregated grain.

MetricLens is a parked side project — not one of the four flagship systems (MetaSignal, RiskFrame, DevPulse, PulseRank). Its purpose is to improve GitHub credibility, blog authority, and interview positioning by demonstrating statistical rigor in a narrow, well-executed open-source library.

2. Product Positioning

Where MetricLens Sits

MetricLens occupies a real but narrow gap in the open-source Python ecosystem. The gap is: automated, quantitative metric movement decomposition from a pandas DataFrame. Most teams perform this analysis manually — slicing every dimension in a spreadsheet or BI tool until a pattern appears. MetricLens packages the decomposition step quantitatively, ranks segments by contribution, and separates mix effects from rate effects in a single library call.

What It Is Not Competing With

Evidently AI — detects data and model drift over time in ML pipelines. MetricLens is for business metric analysis. They do not compete directly; the README should say so explicitly.

Great Expectations — data validation and quality gates for pipelines. MetricLens includes a lightweight profiler for RCA input validation, not a general-purpose pipeline validator.

MetricFlow / dbt — metric definition and semantic layer. MetricLens does not define metrics; it analyzes movement between periods for metrics the user already computes.

Sisu Data (commercial) — automated business RCA as a SaaS product. MetricLens is the open-source, bring-your-own-DataFrame alternative for teams that cannot or will not send data to a SaaS vendor. This is the clearest competitive framing and should appear in the README.

Amplitude / Mixpanel — product analytics SaaS with manual breakdown views. MetricLens automates the decomposition step and produces structured output, not a dashboard.

Non-Overlap With Flagship Systems

MetaSignal answers: did the experiment cause a metric to change? MetricLens answers: where did the metric change and what drove the movement? These are sequential analytical questions and complementary tools. MetricLens does not perform experiment evaluation, CUPED variance reduction, guardrail assessment, or statistical significance testing on contributions. Those belong to MetaSignal.

3. What MetricLens Is / Is Not

MetricLens Is

- A Python library for metric movement decomposition between two time periods
- A tool for quantifying how much each segment of each dimension contributed to a metric delta
- A mix-shift decomposition engine for ratio and average metrics
- A data quality profiler for validating RCA input data before decomposition runs
- A structured report generator producing JSON, Markdown, and HTML output
- A pip-installable package with a stable public API and typed output schema
- Honest: it uses language like "segment contributor," "rate effect," "mix effect," "investigation area"

MetricLens Is Not

- A causal inference tool (MetaSignal owns that domain)
- An experiment evaluation or A/B testing platform
- A real-time or streaming analytics system
- An anomaly detection library
- A time-series forecasting tool
- A BI dashboard or visualization tool
- An ML model or ML pipeline component
- A general-purpose data quality validator (Great Expectations owns that domain)
- A metric definition layer (MetricFlow/dbt own that domain)
- A multi-dimension joint attribution system (Shapley attribution is v1)

Language Policy

Use throughout the codebase, documentation, and reports: "segment contributor," "top contributing segment," "metric movement," "mix effect," "rate effect," "cross term," "investigation area," "decomposition," "signal."

Never use: "cause," "because X caused Y," "causal driver," "proof," "explains the drop" (use "accounts for X% of the movement"), "AI-powered" (there is no AI in v0), "root cause" (use "metric movement decomposition").

4. User Personas

Persona 1 — The Investigating Data Scientist. A DS at a product or e-commerce company. A metric dropped 12% week-over-week. Their manager asked for an RCA before the 9am meeting. They have a daily aggregated reporting DataFrame. They want a ranked breakdown of which segments drove the drop, whether it was a mix shift or a rate shift, and

a formatted report they can share. They do not have time to write groupby loops over five dimensions.

Persona 2 — The Product Analyst. Runs weekly business reviews. Needs to document metric movements with structured evidence. Uses MetricLens to produce consistent, format-stable reports that can be dropped into Notion or Confluence. Values the data quality section because inconsistent data has embarrassed them in reviews before.

Persona 3 — The Portfolio Builder. A data scientist building a professional GitHub portfolio. Uses MetricLens to demonstrate statistical rigor — mix-shift decomposition, honest uncertainty language, clean API design, well-tested library. The project signals systems thinking, not just model training.

Persona 4 — The Interviewer Target. A senior DS or staff ML engineer interviewing at a tech company. Uses MetricLens to demonstrate that they understand the difference between correlation and causation in business analytics, that they know how mix effects and rate effects differ, and that they can build a production-quality open-source library.

5. Use Cases

UC-1: Revenue drop investigation. A D2C e-commerce company reports revenue down 18% in the current week. The analyst runs MetricLens with SumMetric("revenue") over channel, device, city, and product category. MetricLens produces a ranked segment contribution table showing that paid mobile in the US drove 71% of the absolute decline, and a separate table showing the revenue share of that segment changed while total sessions held flat — consistent with a rate shift rather than a mix shift.

UC-2: Conversion rate decomposition. A SaaS product sees conversion rate (signups / visits) drop 2.2pp in the current sprint. The analyst runs MetricLens with RatioMetric("signups", "visits") over traffic source, device, geography, and plan type. The mix-shift decomposition reveals that the rate effect in organic desktop accounts for 1.4pp of the drop while the mix effect from a shift toward mobile traffic accounts for 0.6pp. These are different investigation paths.

UC-3: AOV change. AOV dropped \$8.20 this month. The analyst runs MetricLens with RatioMetric("revenue", "orders", name="aov") over category and channel on pre-aggregated daily data. The mix-shift decomposition shows that high-AOV electronics purchases shifted to a smaller share of orders (mix effect) while per-category AOV held roughly flat (rate effect).

UC-4: Weekly business review report. An analyst runs MetricLens on Monday morning for four metrics (revenue, orders, conversion rate, AOV) over the prior week versus the week before. MetricLens produces four HTML reports and four JSON files. The JSON files are version-controlled. No manual groupby required.

UC-5: Portfolio demonstration. A data scientist runs MetricLens on the synthetic e-commerce demo dataset, explains the mix-shift decomposition in a blog post, and links to the open-source library. In an interview, they explain the cross-term semantics and the design decision to treat AverageMetric as a special case of ratio decomposition.

6. v0 Scope

The following and only the following is in scope for v0.

Metric types: SumMetric, CountMetric, RatioMetric, AverageMetric. No other metric types.

Decomposition engines: Additive decomposition (segment delta, contribution %, share delta) for SumMetric and CountMetric. Three-way mix/rate/cross decomposition for RatioMetric and AverageMetric. No multi-dimension joint decomposition. No Shapley attribution.

Data quality checks: Row count check, date coverage check, null rate check per dimension column, grain check. Four checks only.

Report formats: JSON (primary, stable schema), Markdown (human-readable), HTML (shareable). No PDF, no Excel, no Parquet.

Demo: Synthetic e-commerce / D2C daily aggregated dataset. Grain: date × channel × device × city × category. One scripted scenario. Runnable end-to-end with `python examples/ecommerce_demo.py`. No IPL data.

Infrastructure: pyproject.toml-based packaging, GitHub Actions CI (pytest on push), CHANGELOG.md, METHODOLOGY.md, README.md with actual output numbers from seed=42 run.

Not in v0: CLI, LLM layer, agent, Streamlit, Shapley, anomaly detection, forecasting, streaming, multi-metric joint analysis, statistical significance on contributions, confidence intervals, bootstrap, LiftMap implementation, confidence field.

7. Explicit Non-Goals

These are not deferred — they are explicitly out of scope for v0 and must not be partially implemented, stubbed, or referenced in code as TODOs.

- **LLM/AI layer.** No LLM calls anywhere in the core package or optional extras. The JSON output is designed so a future LLM narrator can consume it — that is an integration point, not an in-scope feature.
- **CLI.** No command-line interface in v0. CLI may be added in v0.2 as an optional extra.
- **Shapley attribution.** Shapley-based dimension attribution is v1. Do not partially implement or stub Shapley.
- **Anomaly detection.** No z-score, no EWMA, no flagging of "unusual" values.
- **Statistical significance.** No p-values, no confidence intervals, no bootstrap on contributions in v0.
- **Forecasting.** MetricLens is retrospective only.
- **Multi-dimension joint analysis.** v0 analyzes one dimension at a time.
- **Streaming or incremental updates.** MetricLens is batch-only.
- **Dashboards or UI.** No frontend of any kind.
- **Causal inference.** No DAG, no instrumental variable, no do-calculus.
- **LiftMap implementation.** LiftMap is a future v2 Opportunity Mode. No liftmap.py, no opportunity.py, no benchmark parameter, no stubs, no TODOs in code. LiftMap concept is documented in Part III of this PRD only.
- **confidence field.** No confidence score, confidence interval, or evidence weight field in any v0 schema.
- **"AI-powered" language.** This phrase must not appear anywhere in the codebase, README, pyproject.toml description, or documentation.

8. Metric Type Design

Abstract Base

```
PYTHON
from abc import ABC, abstractmethod
import pandas as pd

class MetricSpec(ABC):
    @abstractmethod
    def compute(self, df: pd.DataFrame) -> float:
        """Aggregate df to a single scalar metric value."""
    @abstractmethod
    def compute_by_group(self, df: pd.DataFrame, dimension: str) -> pd.DataFrame:
        """Return per-segment aggregation needed for decomposition."""
```

```

@property
@abstractmethod
def decomposition_type(self) -> str:
    """Returns 'additive' or 'ratio'."""
@property
@abstractmethod
def display_name(self) -> str:
    """Human-readable name used in reports."""
@property
def display_unit(self) -> str:
    """Optional unit label for report formatting (e.g. '$', '%', '').
    Defaults to empty string. Override in subclass if needed.
    ReportBuilder uses this for column headers only – never baked
    into schema field names."""
    return ""

```

SumMetric

```

PYTHON
class SumMetric(MetricSpec):
    def __init__(self, col: str, name: str = None, unit: str = ""):
        self.col = col
        self._name = name or f"sum({col})"
        self._unit = unit
    def compute(self, df: pd.DataFrame) -> float:
        return float(df[self.col].sum())
    def compute_by_group(self, df: pd.DataFrame, dimension: str) -> pd.DataFrame:
        return (
            df.groupby(dimension, observed=True)[self.col]
            .sum().reset_index().rename(columns={self.col: "value"})
        )
    @property
    def decomposition_type(self) -> str: return "additive"
    @property
    def display_name(self) -> str: return self._name
    @property
    def display_unit(self) -> str: return self._unit

```

CountMetric

Row count or column count. Internally additive. `col=None` counts rows.

```

PYTHON
class CountMetric(MetricSpec):
    def __init__(self, col: str = None, name: str = "count"):
        self.col = col
        self._name = name
    def compute(self, df: pd.DataFrame) -> float:
        return float(len(df)) if self.col is None else float(df[self.col].count())
    def compute_by_group(self, df: pd.DataFrame, dimension: str) -> pd.DataFrame:
        if self.col is None:
            return df.groupby(dimension, observed=True).size().reset_index(name="value")
        return (
            df.groupby(dimension, observed=True)[self.col]
            .count().reset_index().rename(columns={self.col: "value"})
        )
    @property
    def decomposition_type(self) -> str: return "additive"
    @property
    def display_name(self) -> str: return self._name

```

RatioMetric

Numerator sum divided by denominator sum. Decomposition type: ratio.

PYTHON

```
class RatioMetric(MetricSpec):
    def __init__(self, numerator: str, denominator: str,
                 name: str = None, unit: str = ""):
        self.numerator = numerator
        self.denominator = denominator
        self._name = name or f"{numerator} / {denominator}"
        self._unit = unit
    def compute(self, df: pd.DataFrame) -> float:
        n, d = df[self.numerator].sum(), df[self.denominator].sum()
        if d == 0:
            raise ZeroDenominatorError(
                f"Denominator column '{self.denominator}' sums to zero.")
        return float(n / d)
    def compute_by_group(self, df: pd.DataFrame, dimension: str) -> pd.DataFrame:
        g = df.groupby(dimension, observed=True).agg(
            numerator=(self.numerator, "sum"),
            denominator=(self.denominator, "sum"),
        ).reset_index()
        g["rate"] = g["numerator"] / g["denominator"].replace(0, float("nan"))
        return g
    @property
    def decomposition_type(self) -> str: return "ratio"
    @property
    def display_name(self) -> str: return self._name
    @property
    def display_unit(self) -> str: return self._unit
```

AverageMetric

Average of a numeric column. Internally: $\text{sum}(\text{col}) / \text{count}(\text{col})$. Uses ratio decomposition.

PYTHON

```
class AverageMetric(MetricSpec):
    def __init__(self, column: str, name: str = None, unit: str = ""):
        self.column = column
        self._name = name or f"avg({column})"
        self._unit = unit
    def compute(self, df: pd.DataFrame) -> float:
        return float(df[self.column].mean())
    def compute_by_group(self, df: pd.DataFrame, dimension: str) -> pd.DataFrame:
        g = df.groupby(dimension, observed=True).agg(
            numerator=(self.column, "sum"),
            denominator=(self.column, "count"),
        ).reset_index()
        g["rate"] = g["numerator"] / g["denominator"].replace(0, float("nan"))
        return g
    @property
    def decomposition_type(self) -> str: return "ratio"
    @property
    def display_name(self) -> str: return self._name
    @property
    def display_unit(self) -> str: return self._unit
```

When to Use AverageMetric vs RatioMetric

Use **AverageMetric** when your data is row-level (each row is one event or transaction) and you want to average a numeric column across rows. Example: a DataFrame where each row is one order with an `order_value` column.

Use **RatioMetric for pre-aggregated data**. When your data is already aggregated by date and dimension with `revenue` and `orders` as pre-summed columns, AOV is correctly expressed as `RatioMetric("revenue", "orders", name="aov")`. Using `AverageMetric("revenue")` on pre-aggregated data would average daily revenue values, not per-order revenue — which is incorrect.

The v0 demo dataset uses daily aggregated grain. All demo metrics therefore use `SumMetric` or `RatioMetric`. `AverageMetric` is included in v0 for correctness and row-level event data use cases, but does not appear in the primary demo.

9. Decomposition Methodology

Additive Metrics (*SumMetric, CountMetric*)

Total delta: $\Delta V = V_c - V_b$

For segment i of dimension D :

- Segment delta: $\Delta \blacksquare = V_{c_i} - V_{b_i}$
- Contribution %: $C \blacksquare = \Delta \blacksquare / |\Delta V| \times 100$ (see zero-delta handling below)
- Baseline share: $S_{b_i} = V_{b_i} / V_b \times 100$
- Share delta (pp): $\Delta S_i = S_{c_i} - S_{b_i}$

Contribution sign semantics: When $\Delta V > 0$, signed sum of all $C \blacksquare = +100\%$. When $\Delta V < 0$, signed sum = -100% . Absolute sum can exceed 100% when segments offset each other — this is correct, not a bug.

Mix-shift decomposition is NOT applied to additive metrics. `result.mix_shift()` on an additive metric raises `DecompositionTypeError`.

Zero Total Delta

When $|\Delta V| < 1e-9$:

- `direction = "FLAT"`, `contribution_pct = None` for all segment rows
- `relative_delta_pct` in `ExecutiveSummary` = `None`
- Quality warning added: "Net metric movement is approximately zero. Contribution percentages are undefined and will appear as null."

Zero Baseline Value

When $|baseline_value| < 1e-9$:

- `relative_delta_pct = None`
- `absolute_delta` still computed
- Quality warning added: "Baseline metric value is zero or near-zero. Relative delta % is undefined."

Ratio / Average Metrics (*RatioMetric, AverageMetric*)

Three-way decomposition:

```
Mix effect per segment:  $M \blacksquare = (w_{\blacksquare} - w_{\blacksquare}) \times r_{\blacksquare}$ 
Rate effect per segment:  $R \blacksquare = (r_{\blacksquare} - r_{\blacksquare}) \times w_{\blacksquare}$ 
Cross term per segment:  $X \blacksquare = (w_{\blacksquare} - w_{\blacksquare}) \times (r_{\blacksquare} - r_{\blacksquare})$ 
```

Identity (exact): $\sum M + \sum R + \sum X = \Delta R = R_c - R_b$

Any floating-point violation beyond 1e-10 indicates a bug. The test suite asserts this identity for every test scenario.

New and Disappeared Segments — Corrected Formula

Zero-fill convention: absent segments receive $w = 0$, $r = 0$.

New segment (present in current, absent in baseline):

```
Mix: M = (w - 0) × 0 = 0
Rate: R = (r - 0) × 0 = 0
Cross: X = w × r [positive — full contribution in cross-term]
```

Disappeared segment (present in baseline, absent in current):

```
Mix: M = (0 - w) × r = -w × r [negative]
Rate: R = (0 - r) × w = -r × w [negative]
Cross: X = (0 - w) × (0 - r) = +w × r [positive, NOT zero]
Total: -w × r - r × w + w × r = -w × r ✓
```

The cross-term for a disappeared segment is **positive** and partially offsets the double-counted negative mix and rate effects. The identity holds exactly.

Report footnote: "For segments absent in one period, individual mix/rate/cross values reflect the zero-fill arithmetic convention and are less behaviorally interpretable in isolation. Use the total column and segment_status for interpretation."

Null Dimension Handling

```
PYTHON
working_df = df.copy()
working_df[dimension] = working_df[dimension].fillna("(null)")
```

Original data is never modified. Null rate check runs on original data before fill.

10. Data Quality Design

DataProfiler runs before any decomposition. FAIL-level checks do not block decomposition — they add a bold banner to the report. DataProfiler results are always Section 2 of every report.

Check 1 — Row Count. WARN if ratio outside [0.5, 2.0]. FAIL if outside [0.1, 10.0].

Check 2 — Date Coverage. WARN with missing date list if any date missing. FAIL if >20% of expected dates missing.

Check 3 — Null Rate. WARN if null rate >0% and ≤5%. FAIL if >5%. Runs on original data before null fill.

Check 4 — Grain. WARN if multiple rows per (date, dimension_combination). Event-level data is valid; WARN only.

Period Length Warning. If current and baseline periods differ in calendar days, standalone warning appended: "Volume-based comparisons may be misleading. Rate-based comparisons are not affected."

11. Report Output Design

Every report contains exactly seven sections in this order. Section ordering is not configurable in v0.

Section 1 — Metadata Block. Library version, metric name, metric type, periods, dimensions, run timestamp (UTC), data shape.

Section 2 — Data Quality Summary. One row per check. Columns: check name, status, detail. Banner if any WARN/FAIL.

Section 3 — Executive Summary. Baseline value, current value, absolute delta, relative delta % (null if baseline zero or delta zero), direction, period details. For ratio/average: denominator totals.

Section 4 — Per-Dimension Segment Contribution Tables. One table per dimension. Sorted by labsolute_deltal descending. Mandatory footnote: "Contribution % uses lttotal_deltal as denominator. Signed sum = +100% if metric rose, -100% if metric fell. Absolute sum may exceed 100% when segments offset. Null when net movement is approximately zero."

Section 5 — Mix-Shift Decomposition Tables. Ratio/average metrics only. One table per dimension. Columns: segment, segment_status, mix_effect, rate_effect, cross_term, total, dominant_driver. Totals row with identity check residual. Footnote for new/disappeared segments.

Section 6 — Investigation Areas. At most 5 (dimension, segment) pairs. Ranking: lrate_effectl + lmix_effectl for ratio; labsolute_deltal for additive. Header: "These are suggested starting points for investigation, not confirmed causes."

Section 7 — Interpretation Note. Fixed text, mandatory on every report, cannot be suppressed: "MetricLens computes metric movement decomposition. Contribution is not causation. These results identify where the metric changed and decompose the movement into mix effects and rate effects. They do not explain why those changes occurred. Use this report to prioritize investigation, not to draw conclusions. For causal attribution, conduct a controlled experiment."

12. JSON Schema / Output Contract

The JSON output is the primary format. It is stable, machine-readable, and designed so a future LLM narrator can consume it with no additional computation. `schema_version` is derived from `__version__` major.minor: "0.1" from "0.1.0".

```
PYTHON
@dataclass
class MetadataBlock:
    library_version: str
    metric_name: str
    metric_type: str
    metric_display_unit: str
    current_period_start: str
    current_period_end: str
    current_period_days: int
    baseline_period_start: str
    baseline_period_end: str
    baseline_period_days: int
    dimensions: List[str]
    run_timestamp_utc: str
    total_rows: int
@dataclass
class ExecutiveSummary:
    baseline_value: float
    current_value: float
    absolute_delta: float
    relative_delta_pct: Optional[float] # None if baseline == 0 or delta ≈ 0
    direction: str # "UP" | "DOWN" | "FLAT"
    baseline_denominator: Optional[float] = None
    current_denominator: Optional[float] = None
```

```

denominator_delta: Optional[float] = None
@dataclass
class SegmentRow:
    segment: str
    segment_status: str # "normal"|"new"|"disappeared"|"zero_denominator"
    baseline_value: Optional[float] = None
    current_value: Optional[float] = None
    absolute_delta: Optional[float] = None
    contribution_pct: Optional[float] = None # None when total_delta ≈ 0
    baseline_share_pct: Optional[float] = None
    current_share_pct: Optional[float] = None
    share_delta_pp: Optional[float] = None
    baseline_denominator: Optional[float] = None
    current_denominator: Optional[float] = None
    baseline_rate: Optional[float] = None
    current_rate: Optional[float] = None
    rate_delta: Optional[float] = None # unit-agnostic (not always pp)
    baseline_weight: Optional[float] = None
    current_weight: Optional[float] = None
    weight_delta: Optional[float] = None
@dataclass
class MixShiftRow:
    segment: str
    segment_status: str
    mix_effect: float
    rate_effect: float
    cross_term: float
    total: float
    dominant_driver: str
@dataclass
class DimensionResult:
    dimension: str
    segment_rows: List[SegmentRow]
    mix_shift_rows: Optional[List[MixShiftRow]] = None
    total_mix_effect: Optional[float] = None
    total_rate_effect: Optional[float] = None
    total_cross_term: Optional[float] = None
    decomposition_identity_residual: Optional[float] = None # should be ~0.0
@dataclass
class AnalysisResult:
    metadata: MetadataBlock
    quality_checks: List[QualityCheckResult]
    executive_summary: ExecutiveSummary
    dimension_results: List[DimensionResult]
    investigation_areas: List[InvestigationArea]
    interpretation_note: str
    schema_version: str = "0.1"

```

Field note: `rate_delta_pp` and `weight_delta_pp` from v0 draft renamed to `rate_delta` and `weight_delta`. `_pp` suffix incorrectly implied percentage-point units, which is wrong for dollar-valued metrics. `decomposition_identity_check` renamed to `decomposition_identity_residual` (stores the residual, not a boolean).

13. Package Architecture

```

metriclens/
├── __init__.py
├── _version.py # __version__ = "0.1.0" – single source of truth
├── core.py # MetricLens orchestrator + analyze()
└── metrics.py # MetricSpec, SumMetric, CountMetric, RatioMetric, AverageMetric

```

```

#### period.py # PeriodSpec + period-slicing utilities
#### profiler.py # DataProfiler + CheckResult
#### engine.py # DeltaEngine
#### segments.py # SegmentAnalyzer (null-fill step)
#### decomposition.py # MixShiftAnalyzer (ratio/average only)
#### report.py # ReportBuilder (JSON / Markdown / HTML)
#### schema.py # All output dataclasses
#### exceptions.py # MetricLensError, DecompositionTypeError,
# ZeroDenominatorError, PeriodError, DataQualityFatalError
#### templates/
# #### report.html.j2
# #### report.md.j2
tests/
#### conftest.py
#### test_metrics.py
#### test_profiler.py
#### test_engine.py
#### test_segments.py
#### test_decomposition.py
#### test_report.py
#### data/
#### ecommerce_fixture.csv
examples/
#### ecommerce_demo.py
#### generate_demo_data.py
docs/
#### METHODOLOGY.md
pyproject.toml
README.md
CHANGELOG.md
.github/workflows/ci.yml

```

Dependencies

Core: **pandas**, **numpy**, **jinja2**. Dev extras only: **pytest>=7.0**, **ruff>=0.1**.

```

TOML
[project.optional-dependencies]
dev = ["pytest>=7.0", "ruff>=0.1"]

```

No **llm**, **cli**, or **streamlit** extras in v0.

14. Public API Design

```

PYTHON
lens = MetricLens(
    data=df,
    date_col="date",
    current_period=("2026-04-01", "2026-04-07"),
    baseline_period=("2026-03-25", "2026-03-31"),
    dimensions=["channel", "device", "city", "category"],
)
result = lens.analyze(metric=SumMetric("revenue"))
result = lens.analyze(metric=RatioMetric("orders", "sessions", name="cvr"))
result = lens.analyze(metric=RatioMetric("revenue", "orders", name="aov"))
result.summary()
result.segment_contributions(dimension="channel") # pd.DataFrame
result.mix_shift(dimension="channel") # pd.DataFrame (ratio only)
result.to_json("revenue_rca.json")

```

```
result.to_markdown("revenue_rca.md")
result.to_html("revenue_rca.html")
html_str = result.to_html() # no file path = returns string
```

15. Demo Dataset Plan

Grain: date × channel × device × city × category

Columns: date, channel (5), device (3), city (8), category (5), sessions, orders, revenue, refunds

Scale: ~600 combinations per day, 14 days total (~6,000–8,000 rows). Sparse combinations handled by missing rows, not zero rows.

Scripted scenario: paid_search mobile conversion rate drops from ~3.2% to ~1.8% (rate effect dominant). Session share in that segment also declines 4pp (mix effect, same direction) — producing a large cross-term. Email shows partially offsetting +0.6pp rate improvement. Electronics shows a pure mix shift in AOV.

Generator: `python examples/generate_demo_data.py --seed 42 --output examples/ecommerce_demo.csv`

Demo: `python examples/ecommerce_demo.py` — runs four analyses (SumMetric revenue, SumMetric orders, RatioMetric conversion rate, RatioMetric AOV), prints tables to stdout, writes four HTML reports to `examples/output/`. Under 5 seconds. Deterministic.

16. Testing and Acceptance Gates

All tests runnable with `pytest tests/ -v`. Must pass on Python 3.10, 3.11, and 3.12.

Decomposition Identity Tests

test_identity_basic — assert `|total_mix + total_rate + total_cross - delta_R| < 1e-10`

test_identity_pure_mix — all rates identical, weights shift → `total_rate == 0`, `total_cross == 0`

test_identity_pure_rate — all weights identical, rates change → `total_mix == 0`, `total_cross == 0`

test_identity_new_segment — new segment: `status="new"`, `mix=0`, `rate=0`, `cross=w_c × r_c`

test_identity_disappeared_segment — disappeared segment: `cross=+w_b × r_b` (positive), identity holds

test_zero_denominator_segment — tagged "zero_denominator", excluded from totals, warning present

test_zero_denominator_fatal — overall aggregate denominator = 0 → `ZeroDenominatorError` raised

Additive Tests

test_sum_contribution_signed_sum — signed sum = +100.0 when `delta > 0`, -100.0 when `delta < 0`

test_sum_contribution_absolute_exceeds_100 — offsetting segments → `lcontribution_pctl > 100` for dominant

test_zero_total_delta — `direction="FLAT"`, `contribution_pct=None`, quality warning present

test_zero_baseline_value — `relative_delta_pct=None` in `ExecutiveSummary`, `absolute_delta` still computed

test_mix_shift_on_additive_raises — `SumMetric` result.mix_shift() raises `DecompositionTypeError`

DataProfiler Tests

test_null_rate_warn / test_null_rate_fail — 3% → WARN, 15% → FAIL

test_null_fill_not_modifying_original — original `DataFrame` unchanged after analysis

test_null_segment_appears_in_output — "(null)" appears in `segment_contributions` output

test_missing_dates_warn / test_missing_dates_fail — partial missing → WARN, >20% → FAIL

test_duplicate_grain_warn — duplicate rows → WARN

test_period_length_mismatch_warning — 7-day vs 14-day → warning in profiler output

Report and Schema Tests

test_json_valid — valid JSON, correct top-level keys

test_json_schema_version — `schema_version` == "0.1"

test_json_field_names — `rate_delta` present, `rate_delta_pp` absent

test_contribution_pct_null_in_flat_case — all `contribution_pct` null when `total_delta` ≈ 0

test_relative_delta_pct_null_on_zero_baseline — `relative_delta_pct` null in `executive_summary`

test_html_required_sections — all seven section headers present

test_interpretation_note_always_present — mandatory text in HTML and JSON regardless of quality

Acceptance Gates Before v0.1.0 Release

- All pytest tests pass on Python 3.10, 3.11, 3.12
- `python examples/ecommerce_demo.py` runs under 10 seconds
- README demo numbers match actual --seed 42 output (copy-pasted)
- `pip install metriclens` installs cleanly, no LLM or CLI extras
- `METHODOLOGY.md` contains algebraic decomposition derivation
- `METHODOLOGY.md` contains corrected disappeared segment cross-term derivation
- `rate_delta_pp` does not appear anywhere in codebase
- No `shapley.py`, `liftmap.py`, `opportunity.py`, `cli.py`, `llm.py` in repo

17. README Plan

Section 1 — One-line: "MetricLens is a Python library for metric movement decomposition. It tells you where your metric changed and whether it was driven by mix shift, rate shift, or both."

Section 2 — `pip install metriclens`

Section 3 — Quick start: constructor + `lens.analyze(SumMetric("revenue")) + result.to_html()`

Section 4 — "What MetricLens Is Not": not causal inference, not BI dashboard, not model monitoring, not Sisu Data (MetricLens is the open-source `DataFrame`-native alternative)

Section 5 — Worked example: actual terminal output from `ecommerce_demo.py --seed 42`. Real numbers. Caption: "Actual output from examples/ecommerce_demo.py with seed=42 on daily aggregated e-commerce data."

Section 6 — Metric types table: type, decomposition method, when to use

Section 7 — Report formats: JSON schema keys, Markdown table sample, HTML structure

Section 8 — Links: METHODOLOGY.md, CHANGELOG.md, examples/

Section 9 — "Why MetricLens?": the gap, the design principles (deterministic, no LLM in core, cross-term explicitly reported, honest contribution language)

18. METHODOLOGY.md Plan

Written before implementation begins. Serves as the specification the test suite is written against.

Sections: What MetricLens Does · What It Does Not Do · Metric Types and Decomposition · Additive Decomposition (sign convention, zero-delta, zero-baseline) · Ratio Decomposition — Full Algebraic Derivation · Cross-Term Semantics · New and Disappeared Segments — Corrected Formulas (explicit: "cross-term for a disappeared segment is $+w_b \times r_b$, positive not zero") · Null Dimension Handling · AverageMetric as Ratio (proof + scope note) · Contribution vs. Causation · Limitations · Version History.

19. Roadmap Overview

This section summarises the three-phase release plan. Part II and Part III of this PRD specify v1 and v2 in full.

v0.1.0 (Build-Ready — this document)

Movement Mode only. Four metric types. Additive and ratio decomposition. DataProfiler. ReportBuilder. JSON/Markdown/HTML. pytest suite. Synthetic e-commerce demo.

v0.2 — 4–6 weeks after v0.1.0 release

- CLI: `metriclens rca data.csv --metric revenue ...`
- Bootstrap confidence intervals on contributions (optional, `bootstrap_n` parameter)
- Optional second demo using SaaS/product analytics aggregated data to demonstrate domain generality.

v1 (8–12 weeks after v0.1.0)

- Shapley attribution across dimensions (permutation sampling)
- Multi-metric analysis (`lens.analyze_many(metrics=[...])`)
- Optional LLM narrator (`result.to_narrative(llm_client=client)`)
- HypothesisRanker structured JSON output
- `pip install metriclens[llm]` extra

v2 (12+ weeks after v1)

- LiftMap Mode — Opportunity Detection (fully specified in Part III)
- Streamlit demo application
- Multi-dimension joint decomposition

- Time-series within-period breakdown
- dbt/MetricFlow integration

20. Risks and Traps

Trap 1 — Mix-shift on additive silently produces numbers. Guard at both `result.mix_shift()` and inside `MixShiftAnalyzer.run()`.

Trap 2 — `observed=True` omission. Always required in all groupby calls. Test with categorical dtype dimensions.

Trap 3 — Floating-point identity failure. Tolerance is 1e-10, not 1e-6. Fix the implementation, do not widen the tolerance.

Trap 4 — Disappeared segment cross-term set to zero. The corrected formula is $+w_b \times r_b$ (positive). `test_identity_disappeared_segment` verifies this explicitly.

Trap 5 — Unequal period lengths silently mislead. DataProfiler must warn. Executive summary must display period lengths explicitly.

Trap 6 — Zero total delta not handled. `contribution_pct` must be None, not NaN that silently propagates.

Trap 7 — Original DataFrame modified by null fill. Fill on a copy only.

Trap 8 — README numbers manually typed. All numbers must come from an actual `--seed 42` run.

Trap 9 — LLM extras added to v0. No LLM extras in v0 pyproject.

Trap 10 — `rate_delta_pp` field name persisting. Old name must be completely absent. `test_json_field_names` asserts its absence.

Trap 11 — Shapley or LiftMap stub in v0. No empty files, no TODO comments, no placeholders.

Trap 12 — "AI-powered" language anywhere. Must not appear in README, pyproject description, or documentation.

21. Interview Positioning

The One-Line Summary: "MetricLens is an open-source Python library I built for metric movement decomposition — it separates mix effects from rate effects using the standard three-way ratio decomposition, quantifies per-segment contributions, and explicitly reports the cross-term that most manual analyses discard."

What It Demonstrates: Understanding of the mix-shift decomposition identity and its algebraic derivation. Ability to design a clean, extensible Python library with a well-defined public API. Awareness of the difference between contribution and causation. Production-quality open-source habits. The judgment to build something narrow and correct rather than broad and vague.

How to Use It in Interviews: Explain the cross-term for disappeared segments and why it is positive. Explain the contribution vs. causation distinction. Describe `METHODOLOGY.md` as written-before-implementation — signals a disciplined engineering approach.

How It Complements the Flagship Systems: MetaSignal answers: did the experiment cause a metric to move? MetricLens answers: where did the metric move? Sequential tools in the same analytical workflow.

22. Final Build Checklist

Core

- MetricSpec abstract base with display_unit property (default "")
- SumMetric, CountMetric, RatioMetric, AverageMetric — all implemented
- observed=True in all groupby calls
- ZeroDenominatorError on zero aggregate denominator in RatioMetric

Data Profiling

- All four checks implemented
- Null rate check runs on original data before null fill
- Internal null fill on copy only — original unchanged
- Period length mismatch warning

Decomposition

- Additive signed sum = $\pm 100\%$ (correct sign)
- contribution_pct = None when $|total_delta| < 1e-9$
- relative_delta_pct = None when baseline_value == 0
- Ratio identity holds within $1e-10$ for all test scenarios
- Disappeared segment cross-term = $+w_b \times r_b$ (positive)
- New segment: mix=0, rate=0, cross= $w_c \times r_c$
- dominant_driver correct for all four cases

Schema

- rate_delta present (not rate_delta_pp)
- weight_delta present (not weight_delta_pp)
- rate_delta_pp absent from entire codebase
- contribution_pct typed as Optional[float]
- relative_delta_pct typed as Optional[float]
- decomposition_identity_residual (not decomposition_identity_check)
- schema_version = "0.1" derived from __version__ major.minor

Tests

- test_identity_disappeared_segment: cross= $+w_b \times r_b$ (positive)
- test_zero_total_delta: direction=FLAT, contribution_pct=None
- test_zero_baseline_value: relative_delta_pct=None
- test_null_fill_not_modifying_original
- test_json_field_names: rate_delta present, rate_delta_pp absent
- All tests pass on Python 3.10, 3.11, 3.12

Demo and Documentation

- ecommerce_demo.py uses SumMetric and RatioMetric only
- README demo numbers match actual --seed 42 output
- METHODOLOGY.md contains corrected disappeared segment cross-term
- pyproject.toml has dev extras only (no llm, cli, streamlit)
- No shapley.py, liftmap.py, opportunity.py, cli.py, llm.py in repo
- GitHub Actions CI passes on push

PART II — MOVEMENT MODE v1 EXPANSION

23. v1 Scope Overview

v1 expands Movement Mode without changing its mathematical core or output schema. All v0.1 decomposition logic, schema field names, and report structure are preserved. v1 adds tooling and optional layers on top of a stable foundation.

v1 adds:

- CLI (`metriclens rca`) for running analyses from the terminal without writing Python
- Bootstrap confidence intervals on contributions (optional, disabled by default)
- Shapley attribution across dimensions — which dimensions matter most
- Multi-metric support (`lens.analyze_many(metrics=[...])`)
- Optional LLM narrator (requires `pip install metriclens[llm]`)
- HypothesisRanker structured JSON output for LLM consumption

v1 does not change:

- Any decomposition formula or mathematical identity
- Any v0.1 schema field names
- Any v0.1 report section structure
- v0.1 public API (`analyze()`, `segment_contributions()`, `mix_shift()`, `to_json/md/html()`)

24. CLI Design

The v1 CLI uses Click. Entry point: `metriclens`. Primary subcommand: `rca`. The CLI is an optional extra (`pip install metriclens[cli]`).

```
metriclens rca data.csv \
--metric revenue \
--metric-type sum \
--date-col date \
--current 2026-04-01:2026-04-07 \
--baseline 2026-03-25:2026-03-31 \
--dimensions channel,device,city \
--out report.html \
--format html
```

Arguments:

- `data` — path to CSV file
- `--metric` — column name for the metric
- `--metric-type` — `sum` | `count` | `ratio` | `average`
- `--numerator` / `--denominator` — required when `--metric-type ratio`
- `--date-col` — date column name (default: "date")
- `--current` — current period as `YYYY-MM-DD:YYYY-MM-DD`
- `--baseline` — baseline period as `YYYY-MM-DD:YYYY-MM-DD`
- `--dimensions` — comma-separated list of dimension columns
- `--out` — output file path (default: `rca_output.html`)

- `--format` — `html` | `markdown` | `json` (default: `html`)
- `--bootstrap-n` — integer, enables bootstrap CIs if `> 0` (default: `0`)

Error handling: All `MetricLensErrors` surface as clean CLI error messages with exit code 1. No Python tracebacks shown to end users unless `--debug` flag is set.

CLI module: `metriclens/cli.py`. Not imported in `__init__.py`. Import is guarded — if Click is not installed, `ImportError` is raised with message: "CLI requires metriclens[cli]. Install with: `pip install metriclens[cli]`".

v1 pyproject addition:

```
TOML
[project.optional-dependencies]
dev = ["pytest>=7.0", "ruff>=0.1"]
cli = ["click>=8.0"]
llm = ["openai>=1.0", "anthropic>=0.20"]
```

25. Bootstrap Confidence Intervals

Bootstrap CIs on contributions are optional and disabled by default. They are available for both additive and ratio metrics but most meaningful for ratio metrics.

API:

```
PYTHON
result = lens.analyze(
    metric=RatioMetric("orders", "sessions", name="cvr"),
    bootstrap_n=1000, # number of bootstrap samples, 0 = disabled
    bootstrap_ci=0.95, # confidence level, default 0.95
)
```

Method: Bootstrap resamples rows within each period independently (with replacement) `bootstrap_n` times. For each sample, the full decomposition runs. The empirical distribution of each output value produces the CI bounds.

Fields added to SegmentRow when `bootstrap_n > 0`:

```
PYTHON
contribution_pct_ci_lower: Optional[float] = None
contribution_pct_ci_upper: Optional[float] = None
mix_effect_ci_lower: Optional[float] = None
mix_effect_ci_upper: Optional[float] = None
rate_effect_ci_lower: Optional[float] = None
rate_effect_ci_upper: Optional[float] = None
```

Non-goals: Bootstrap CIs are empirical intervals, not statistical significance tests. MetricLens does not report p-values or make significance decisions. The report includes a mandatory footnote: "Bootstrap CIs are empirical, not frequentist significance tests. A wide CI indicates high sampling variability in that segment, not that the decomposition is wrong."

Implementation note: Bootstrap resampling uses `numpy.random.default_rng(seed)` for determinism when a seed is passed. Default: non-deterministic (no seed). `numpy` is already a core dependency — no new requirements.

26. Shapley Attribution Across Dimensions

Shapley attribution answers: which *dimension* explains the most metric movement? This is distinct from which segment within a dimension explains the most movement.

Problem: When running MetricLens over multiple dimensions, the analyst sees a per-dimension contribution table but has no cross-dimension comparison. Shapley attribution provides a fair allocation of total movement across dimensions.

Method: For K dimensions, the Shapley value of dimension d is its average marginal contribution to the total metric movement explanation across all permutations of dimensions. For $K \leq 4$, all $K!$ permutations are evaluated exactly. For $K > 4$, permutation sampling approximation with `n_permutations` samples (default 200).

API:

```
PYTHON
shapley_result = result.shapley_attribution(n_permutations=200)
shapley_result.dimension_values # pd.DataFrame: dimension, shapley_value, pct_explained
shapley_result.to_json()
```

New schema dataclasses (v1 schema.py):

```
PYTHON
@dataclass
class DimensionShapleyValue:
    dimension: str
    shapley_value: float # absolute value units
    pct_explained: float # |shapley_value| / |total_delta| × 100
@dataclass
class ShapleyResult:
    metric_name: str
    total_delta: float
    dimension_values: List[DimensionShapleyValue]
    method: str # "exact" | "permutation_sampling"
    n_permutations: int
    schema_version: str
```

Report section addition (v1 Section 4.5): Between Section 4 and Section 5, v1 reports add an optional "Dimension Attribution" section when Shapley is computed. Shows ranked bar-style summary: which dimension is most explanatory.

Non-goals: Shapley per segment (not v1 — computationally intensive and less interpretable at the segment level). Shapley with interactions (not v1).

27. Multi-Metric Support

v1 adds a convenience method to run multiple metrics in one call:

```
PYTHON
results = lens.analyze_many(
    metrics=[
        SumMetric("revenue"),
        SumMetric("orders"),
        RatioMetric("orders", "sessions", name="cvr"),
        RatioMetric("revenue", "orders", name="aov"),
    ]
)
# returns List[AnalysisResult]
for result in results:
    result.to_html(f"{result.metadata.metric_name}_rca.html")
```

`analyze_many` is a convenience wrapper over `analyze()`. Each metric is analyzed independently. No cross-metric correlation is computed in v1. Results maintain the same `AnalysisResult` schema. DataProfiler runs once and is shared across all metrics in the batch.

Schema change (v1): `MetadataBlock` gains `batch_run_id: Optional[str] = None` — a UUID assigned when `analyze_many` is called, allowing JSON files from the same batch to be linked.

28. Optional LLM Narrator

The LLM narrator reads `to_json()` output and produces a structured prose summary. The LLM performs no calculations — it only narrates the evidence in the JSON.

API:

```
PYTHON
narrative = result.to_narrative(
    llm_client=anthropic.Anthropic(), # or openai.OpenAI()
    model="claude-3-5-sonnet-20241022", # default for Anthropic
    max_tokens=800,
)
print(narrative) # returns string
```

System prompt structure: The narrator is given: the JSON output, a strict instruction set (no calculations, only narrate what is in the JSON, use honest contribution language, never say "caused"), and an output format (three paragraphs: what changed, where it changed, recommended investigation areas).

Paragraph structure:

1. *What changed:* metric name, direction, absolute delta, relative delta, period dates.
2. *Where it changed:* top two to three contributing segments by absolute delta or rate effect. Use phrasing: "The {dimension}={segment} segment contributed {contribution_pct}% of the total movement."
3. *Investigation areas:* the top two investigation areas from Section 6 of the JSON. Phrasing: "The suggested starting point for investigation is {segment} in the {dimension} dimension."

Guards: If the JSON `direction` is "FLAT", the narrator is instructed to report that the metric was flat and no contribution ranking is meaningful. If any quality check is FAIL, the narrator leads with: "Note: one or more data quality checks failed. Review the data quality section before acting on these findings."

Installation: `pip install metriclens[llm]`. If called without `[llm]` extras installed, raises `ImportError` with install instruction. LLM module is `metriclens/narrator.py`, not imported in `__init__.py`.

29. v1 Package Architecture Changes

New files added in v1:

```
metriclens/
  cli.py # Click CLI — import guarded, not in __init__.py
  narrator.py # LLM narrator — import guarded, not in __init__.py
  shapley.py # ShapleyAnalyzer — exported in __init__.py
  bootstrap.py # BootstrapEngine — used internally by core.py
```

Schema additions in `schema.py`:

- `DimensionShapleyValue`, `ShapleyResult`

- CI fields on `SegmentRow`: `contribution_pct_ci_lower`, `contribution_pct_ci_upper`, `mix_effect_ci_lower`, `mix_effect_ci_upper`, `rate_effect_ci_lower`, `rate_effect_ci_upper`
- `batch_run_id`: `Optional[str]` on `MetadataBlock`

All v0.1 fields preserved. No v0.1 field removed or renamed.

`_version.py` bumped to `__version__ = "1.0.0"`. `schema_version` becomes "1.0".

30. v1 Build Checklist

CLI

- `metriclens rca` runs correctly on `ecommerce_demo.csv` with all argument combinations
- `--metric-type ratio` requires `--numerator` and `--denominator`
- `MetricLensErrors` surface as clean CLI messages, exit code 1
- CLI not imported in `__init__.py`; `ImportError` message if click not installed
- `pip install metriclens[cli]` installs click dependency

Bootstrap

- `bootstrap_n=0` (default) produces identical output to v0.1
- `bootstrap_n=1000` produces non-null CI fields in JSON
- Bootstrap footnote present in report when `bootstrap_n > 0`
- Deterministic when seed is provided

Shapley

- Exact computation for $K \leq 4$ dimensions (verified against brute force)
- Permutation sampling for $K > 4$ (verify convergence at `n_permutations=500`)
- `ShapleyResult` JSON is valid and `schema_version` is correct
- `result.shapley_attribution()` raises if only one dimension analyzed

Multi-Metric

- `analyze_many` returns correct `AnalysisResult` per metric
- `DataProfiler` runs once per batch, not once per metric
- `batch_run_id` consistent across all results in the same batch call

LLM Narrator

- `to_narrative()` without `[llm]` raises `ImportError` with install instructions
- Narrative output contains three paragraphs
- No calculations in LLM prompt — JSON evidence only
- Flat case handled: narrative says metric was flat
- FAIL quality check: narrative leads with data quality caveat

v1 Schema Compatibility

- All v0.1 fields still present in v1 JSON output
- No v0.1 field renamed or removed
- `schema_version = "1.0"` in v1 JSON

PART III — LIFTMAP MODE (v2.0)

31. LiftMap Product Purpose

LiftMap is the Opportunity Detection Mode inside MetricLens. It answers a different question from Movement Mode: not "what changed?" but "where is the upside?"

Movement Mode is retrospective — it decomposes a metric change that already happened between two periods. LiftMap is current-period oriented — given the current distribution of segment rates, it identifies which segments have the most room for improvement relative to a benchmark, weighted by their volume.

LiftMap does not predict future performance and does not claim causality. It identifies segments where the actual rate is below a benchmark, weighted by their denominator volume. A segment with a large denominator and a large rate gap has a high opportunity signal — it is a high-priority candidate for investigation or experiment targeting.

The core formula is simple and transparent: `opportunity_signal = denominator_volume * max(benchmark_rate - segment_rate, 0)`. This rewards both volume (denominator) and gap (benchmark gap). A large-volume segment that is just slightly below benchmark produces moderate signal. A small-volume segment far below benchmark also produces moderate signal. Only large-volume, large-gap segments produce high signal.

LiftMap is only meaningful for ratio and average metrics (conversion rate, AOV, CTR, retention). It is not applicable to additive metrics (total revenue, total orders) where there is no per-segment rate concept meaningful for opportunity comparison.

Both Movement Mode and LiftMap share the same `MetricSpec`, `SegmentAnalyzer`, `DataProfiler`, `ReportBuilder`, and output formats. LiftMap is a second analysis mode on the same `MetricLens` object, not a separate library.

32. LiftMap User Personas

Persona 1 — The Growth Analyst. Wants to find which channels or devices have the most room to improve conversion rate before running an A/B test. LiftMap gives a ranked candidate list with opportunity signal quantified — the analyst does not need to manually calculate rate gaps across every segment.

Persona 2 — The Product Manager. Runs weekly growth reviews. Asks: given our conversion rate goal, which segments are furthest from target and have the most volume to move? LiftMap gives an opportunity-weighted ranking that directly answers the question: where will improvement have the largest impact?

Persona 3 — The Experimentation Designer. Designs targeted A/B tests. LiftMap's opportunity ranking helps prioritize which segment to target first. High opportunity signal is a proxy for high expected experiment value: large denominator means the experiment will have statistical power; large rate gap means there is room to detect improvement.

Persona 4 — The Portfolio Builder. Demonstrates analytical depth beyond "what broke." Two complementary analytical modes in one library: what changed (Movement Mode), and where to focus improvement effort (LiftMap). This breadth signals a practitioner who thinks in terms of the full analytical workflow, not just post-hoc diagnosis.

33. LiftMap Use Cases

UC-L1: Conversion rate opportunity by channel. A SaaS product's overall conversion rate (signups / visits) is 3.1%. The analyst runs LiftMap over the channel dimension. Organic search has a rate of 1.8% and 45,000 visits. Paid search has a rate of 4.9% and 30,000 visits. Benchmark = 3.1%. Opportunity signals: organic = $45,000 \times (3.1\% - 1.8\%) = 585$; paid = 0 (above benchmark). LiftMap identifies organic search as the primary opportunity segment.

UC-L2: AOV opportunity by product category. AOV benchmark is \$82. Electronics AOV is \$94 (above benchmark). Apparel AOV is \$61 with 8,200 orders. Home AOV is \$71 with 6,500 orders. Opportunity: apparel = $8,200 \times (82 - 61) = 172,200$; home = $6,500 \times (82 - 71) = 71,500$. LiftMap ranks apparel as the highest AOV opportunity by volume-weighted gap.

UC-L3: Retention rate by acquisition cohort. Monthly retention rates vary by cohort. The analyst runs LiftMap over acquisition cohorts. High-volume recent cohorts that are below the 30-day retention benchmark are ranked highest for intervention targeting.

UC-L4: Weekly opportunity monitoring. The analyst runs LiftMap every Monday alongside the Movement Mode report. Movement Mode shows what shifted; LiftMap shows where the current distribution of rates represents the largest gap from benchmark. Together they answer the full Monday morning question: what happened, and where should we focus?

34. LiftMap Inputs

LiftMap takes the same **MetricLens** object as Movement Mode. It requires:

- A **RatioMetric** or **AverageMetric** (required — **DecompositionTypeError** raised for **SumMetric** or **CountMetric**)
- A dimension to analyze
- Optional: **benchmark_rate** (float, defaults to population average rate in the analysis period)
- Optional: **outperformance_mode** (bool, default False — when True, also compute outperformance signal)
- Optional: **period** — "current" (default) or "baseline" — which period's data to analyze

Period selection: LiftMap analyzes a single period. Both current and baseline data are still loaded for DataProfiler checks, but the decomposition runs on only one period. Default is "current". The analyst can run LiftMap on "baseline" to see what the opportunity distribution looked like in the prior period.

Benchmark rate: When **benchmark_rate=None** (default), MetricLens computes the population average rate from the analysis period: $R = \sum N_i / \sum D_i$. When a user-provided **benchmark_rate** is given, it is used directly and tagged **benchmark_source = "user_provided"**. A user-provided benchmark is useful when the analyst has an external target (e.g., industry average conversion rate) or wants to measure against a goal rather than the current population average.

35. LiftMap Scoring Methodology

Core Formulas

For a given dimension with segments $i = 1 \dots K$ in the analysis period:

```
Population rate:  $R = \sum N_i / \sum D_i$  (default benchmark)
Per-segment rate:  $r_i = N_i / D_i$ 
Rate gap:  $g_i = \text{benchmark\_rate} - r_i$ 
Opportunity signal:  $O_i = D_i \times \max(g_i, 0)$ 
Outperformance signal:  $P_i = D_i \times \max(r_i - \text{benchmark\_rate}, 0)$ 
```

Segments at or above benchmark: $O_{\text{seg}} = 0$, $P_{\text{seg}} > 0$.

Segments below benchmark: $O_{\text{seg}} > 0$, $P_{\text{seg}} = 0$.

Exactly at benchmark: both = 0.

Properties

- opportunity_signal is in units of (denominator_units × rate_units). For conversion rate: sessions × rate_gap (dimensionless). For AOV: orders × \$gap.
- The sum of opportunity signals across all below-benchmark segments is the total "opportunity volume-gap product." It is not a dollar value or a predicted uplift — it is a ranking index.
- opportunity_rank: 1 = highest opportunity signal. Segments with $O_{\text{seg}} = 0$ receive **opportunity_rank = None**.
- For zero-denominator segments: $O_{\text{seg}} = \text{None}$, excluded from ranking.

Interpretation Guards

The scoring methodology is transparent and non-predictive. High opportunity signal does not imply:

- That improvement is achievable (there may be structural reasons for the rate gap)
- That the rate gap is caused by something actionable
- That targeting this segment will yield the computed signal in business value
- That this segment should receive automatic treatment or resource reallocation

High opportunity signal implies: this segment has high volume and a large gap from benchmark. It is a high-priority candidate for investigation. Whether the gap is actionable requires further analysis, qualitative investigation, or a controlled experiment.

36. LiftMap API Design

```
PYTHON
lift_result = lens.lift(
    metric=RatioMetric("orders", "sessions", name="conversion_rate"),
    dimension="channel",
    period="current", # "current" | "baseline"
    benchmark_rate=None, # defaults to population average
    outperformance_mode=False, # if True, also compute outperformance_signal
)
```

LiftResult Methods

```
PYTHON
lift_result.summary()
# Prints top 5 opportunity segments to stdout with opportunity_signal values
lift_result.opportunity_ranking(dimension="channel")
# Returns pd.DataFrame sorted by opportunity_signal descending
# Includes: segment, denominator_volume, segment_rate, benchmark_rate,
# rate_gap, opportunity_signal, opportunity_rank
lift_result.to_json("lift_channel.json")
lift_result.to_markdown("lift_channel.md")
lift_result.to_html("lift_channel.html")
# No file path → returns string:
json_str = lift_result.to_json()
```

Error Handling

- **SumMetric** or **CountMetric** → **DecompositionTypeError**: "LiftMap requires a ratio or average metric. Additive metrics (SumMetric, CountMetric) do not have a per-segment rate concept suitable for benchmark comparison."
- Zero-denominator overall → **ZeroDenominatorError** (same as Movement Mode)
- Invalid **benchmark_rate** type → **ValueError**

37. LiftMap Report Output

LiftMap reports contain exactly seven sections, parallel to Movement Mode. Section ordering is not configurable.

Section 1 — Metadata Block. Same fields as Movement Mode, plus: **benchmark_rate**, **benchmark_source** ("population_average" or "user_provided"), **analysis_period** ("current" or "baseline"), **outperformance_mode** (bool).

Section 2 — Data Quality Summary. Same four DataProfiler checks as Movement Mode, plus one LiftMap-specific check: Benchmark Coverage Check (WARN if all segments are above benchmark, or fewer than 2 segments are below benchmark — zero opportunity is a valid finding, not an error).

Section 3 — Executive Summary. Analysis period metric value, **benchmark_rate**, **benchmark_source**, count of segments below benchmark, count above benchmark, total opportunity signal (sum across all segments), top opportunity segment (dimension + segment + opportunity_signal).

Section 4 — Segment Opportunity Rankings. One table per dimension. Columns: segment, segment_status, denominator_volume, segment_rate, benchmark_rate, rate_gap, opportunity_signal, opportunity_rank. Sorted by opportunity_signal descending (zeros at bottom). Mandatory footnote: "Opportunity signal = denominator_volume × max(benchmark_rate – segment_rate, 0). A high signal identifies a high-volume segment with a rate gap. It is a ranking index, not a predicted business value."

Section 5 — Outperformance Rankings. Appears only when **outperformance_mode=True**. Columns: segment, denominator_volume, segment_rate, benchmark_rate, rate_gap, outperformance_signal, outperformance_rank. Sorted by outperformance_signal descending. Footnote: "Outperformance signal identifies segments performing above benchmark, weighted by volume. These segments may be worth studying for positive learnings."

Section 6 — Investigation Areas. At most 5 segments with highest opportunity_signal. Mandatory header: "The following segments are ranked by opportunity signal. A high opportunity signal indicates a high-volume segment with a rate gap relative to the benchmark. These are suggested starting points for investigation or experiment targeting. They are not confirmed causes of underperformance."

Section 7 — Interpretation Note. Fixed text, mandatory, cannot be suppressed: "LiftMap computes opportunity signals based on denominator volume and rate gaps relative to a benchmark. A high opportunity signal identifies a high-volume segment where rate is below benchmark. It does not predict that improvement is achievable, does not claim the gap is caused by anything specific, and does not recommend automatic business decisions. Use LiftMap output to prioritize investigation and experiment design. For causal attribution, conduct a controlled experiment."

38. LiftMap JSON Schema

```
PYTHON
@dataclass
class LiftMetadataBlock:
    library_version: str
    metric_name: str
```

```

metric_type: str
metric_display_unit: str
analysis_period: str # "current" | "baseline"
period_start: str
period_end: str
period_days: int
benchmark_rate: float
benchmark_source: str # "population_average" | "user_provided"
outperformance_mode: bool
dimensions: List[str]
run_timestamp_utc: str
total_rows: int
@dataclass
class LiftSegmentRow:
    segment: str
    segment_status: str # "normal" | "zero_denominator"
    denominator_volume: float
    segment_rate: Optional[float] # None if zero_denominator
    benchmark_rate: float
    rate_gap: float # benchmark_rate - segment_rate
    opportunity_signal: float #  $D \times \max(\text{rate\_gap}, 0)$ 
    opportunity_rank: Optional[int] # None if opportunity_signal == 0
    outperformance_signal: Optional[float] = None
    outperformance_rank: Optional[int] = None
@dataclass
class LiftDimensionResult:
    dimension: str
    benchmark_rate: float
    benchmark_source: str
    segment_rows: List[LiftSegmentRow]
    total_opportunity_signal: float # sum of all opportunity_signals
    total_opportunity_volume: float # sum of  $D$  for below-benchmark segments
    below_benchmark_count: int
    above_benchmark_count: int
@dataclass
class LiftInvestigationArea:
    rank: int
    dimension: str
    segment: str
    opportunity_signal: float
    denominator_volume: float
    rate_gap: float
    description: str
@dataclass
class LiftResult:
    metadata: LiftMetadataBlock
    quality_checks: List[QualityCheckResult] # reused from Movement Mode
    dimension_results: List[LiftDimensionResult]
    investigation_areas: List[LiftInvestigationArea]
    interpretation_note: str
    schema_version: str = "2.0"

```

Design notes: `LiftResult` uses a separate `schema_version = "2.0"` corresponding to the v2 package release. `QualityCheckResult` is imported from `schema.py` — same dataclass as Movement Mode. `LiftMetadataBlock` is separate from `MetadataBlock` because the period structure differs (single period vs. two periods).

39. LiftMap Data Quality Reuse

LiftMap reuses the same `DataProfiler` as Movement Mode. All four standard checks apply to the analysis-period data:

- Row count check (against the other period's row count)
- Date coverage check (for the analysis period)
- Null rate check (per dimension column)
- Grain check (duplicate date-dimension rows in analysis period)

LiftMap-specific quality check — Benchmark Coverage Check:

```
Check L1 — Benchmark Coverage.
WARN if all segments are above benchmark: "No below-benchmark segments found.
All segments perform above benchmark rate {R:.4f}. Opportunity signal = 0 for all
segments. This is a valid finding — the metric is performing above average across
all segments in this dimension."
WARN if fewer than 2 segments are below benchmark: "Only {n} segment(s) below
benchmark. Opportunity ranking has limited discriminatory power."
Status: WARN (never FAIL — zero opportunity is a valid business finding).
```

Period length note: LiftMap analyzes a single period. The standard period length mismatch warning only applies if both periods were provided. It is adjusted to: "Current period is {n} days. Baseline period is {m} days. LiftMap analyzes the {analysis_period} period only. Period length does not affect rate comparisons."

40. LiftMap Non-Goals

These are explicitly out of scope for v2 LiftMap. They must not be partially implemented, stubbed, or referenced in LiftMap code.

- **Additive metric LiftMap.** SumMetric and CountMetric do not have a per-segment rate concept suitable for benchmark gap analysis. DecompositionTypeError is raised.
- **Predictive uplift.** LiftMap does not model or predict how much a segment's rate would improve if addressed. opportunity_signal is a ranking index, not a predicted gain.
- **Causal inference.** LiftMap does not explain why a segment underperforms. It identifies candidates for investigation.
- **Automated targeting or resource allocation.** LiftMap is a decision-support tool. It does not trigger campaigns, budget shifts, or experiments.
- **Multi-dimension joint LiftMap.** v2 analyzes one dimension at a time, same as Movement Mode v0.
- **Time-series trending of opportunity_signal.** No trend analysis of how opportunity signals evolve over time.
- **Confidence intervals on opportunity_signal.** No bootstrap on LiftMap outputs in v2.
- **LLM narrator for LiftMap.** The LLM narrator (v1 Movement Mode) is not ported to LiftMap in v2. This is v3+ territory.
- **Benchmark from external data sources.** User-provided benchmark_rate must be a scalar float. No industry benchmark data integration.
- **Segment groupings or hierarchies.** One level of dimension granularity only.

41. LiftMap Testing

All LiftMap tests runnable with `pytest tests/test_lift.py -v`. Must pass on Python 3.10, 3.11, 3.12.

Scoring Tests (highest priority)

test_lift_opportunity_signal_formula — known DataFrame, manually compute expected $O_{\text{seg}} = D_{\text{seg}} \times \max(\text{benchmark} - r_{\text{seg}}, 0)$ for each segment, assert MetricLens matches exactly.

test_lift_outperformance_signal_formula — outperformance_mode=True, assert $P_{\text{seg}} = D_{\text{seg}} \times \max(r_{\text{seg}} - \text{benchmark}, 0)$ for each segment.

test_lift_opportunity_signal_zero_above_benchmark — segments above benchmark → opportunity_signal = 0. Assert outperformance_signal > 0 for those segments (when outperformance_mode=True).

test_lift_default_benchmark_is_population_average — benchmark_source = "population_average", benchmark_rate equals manually computed $\Sigma N / \Sigma D$ within 1e-10.

test_lift_custom_benchmark — user-provided benchmark_rate used, benchmark_source = "user_provided". Assert benchmark_rate in output matches user input exactly.

test_lift_opportunity_rank_ordering — segments with higher opportunity_signal have lower rank numbers. Assert rank is monotonically decreasing as opportunity_signal increases.

test_lift_zero_signal_rank_none — segments with opportunity_signal == 0 have opportunity_rank = None.

Edge Case Tests

test_lift_all_above_benchmark — all segments above benchmark → all opportunity_signal = 0, WARN quality check, benchmark_coverage check present in quality_checks, no error raised.

test_lift_all_below_benchmark — all segments below benchmark → no outperformance_signal > 0, all opportunity_signal > 0.

test_lift_zero_denominator_segment — segment with zero denominator tagged "zero_denominator", excluded from totals, warning present.

test_lift_zero_denominator_fatal — overall aggregate denominator = 0 → ZeroDenominatorError raised.

test_lift_additive_raises — SumMetric → DecompositionTypeError with correct message.

test_lift_single_segment — DataFrame with only one segment in the dimension → graceful output, opportunity_rank = 1 if below benchmark, None if above.

Period Selection Tests

test_lift_current_period — period="current" uses current period data (default). Assert segment rates match current period manually.

test_lift_baseline_period — period="baseline" uses baseline period data. Assert segment rates match baseline period manually.

Report and Schema Tests

test_lift_json_valid — valid JSON, correct top-level keys: metadata, quality_checks, dimension_results, investigation_areas, interpretation_note, schema_version.

test_lift_schema_version — schema_version == "2.0".

test_lift_interpretation_note_present — mandatory interpretation note text present in JSON, Markdown, and HTML outputs.

test_lift_no_movement_mode_schema_fields — LiftResult JSON does not contain Movement Mode fields (absolute_delta, mix_effect, rate_effect, cross_term). Schema separation is clean.

test_lift_html_seven_sections — HTML output contains all seven required section headers.

test_lift_outperformance_section_absent_by_default — when `outperformance_mode=False`, outperformance section not present in report.

test_lift_outperformance_section_present_when_enabled — when `outperformance_mode=True`, outperformance section present.

DataProfiler Reuse Tests

test_lift_profiler_benchmark_coverage_warn_all_above — all segments above benchmark → WARN in quality_checks.

test_lift_profiler_standard_checks_run — null rate, grain, and date coverage checks still run in LiftMap mode.

42. LiftMap Risks

Risk L1 — Opportunity signal misread as predicted uplift. Analysts familiar with uplift modeling may interpret high `opportunity_signal` as a predicted performance gain. The mandatory interpretation note on every report guards against this. The word "predicted" must never appear in LiftMap outputs.

Risk L2 — LiftMap misused for automated targeting. Without the interpretation note, a product team might automatically allocate budget or trigger campaigns for high-opportunity segments. The non-goal section and report language are the guards. Additionally: the `investigation_areas` header uses "suggested starting points for investigation or experiment targeting, not confirmed causes."

Risk L3 — Additive metric passed to LiftMap. `SumMetric("revenue")` does not have a per-segment rate concept meaningful for benchmark comparison. `DecompositionTypeError` is raised with clear message. Do not silently compute a nonsensical result.

Risk L4 — All segments above benchmark. Produces empty opportunity ranking. MetricLens handles this gracefully: `total_opportunity_signal = 0.0`, WARN quality check, no error raised. Report Section 4 shows all segments with `opportunity_signal = 0` and includes a note: "All segments perform above the benchmark rate. This is a valid finding."

Risk L5 — User-provided benchmark_rate is unreasonable. A rate far outside the range of observed segment rates may produce misleading rankings. MetricLens does not validate the range of a user-provided benchmark. WARN is added if `benchmark_rate < min(segment_rates)` or `benchmark_rate > max(segment_rates)`: "User-provided benchmark rate {value} is outside the range of observed segment rates [{min}, {max}]. Rankings may not be meaningful."

Risk L6 — Confusing Movement Mode and LiftMap outputs. The API separation is `lens.analyze()` vs `lens.lift()`. The JSON schemas are distinct (`AnalysisResult` vs `LiftResult`). `schema_version` differs ("0.1"/"1.0" vs "2.0"). These guards ensure the two modes cannot be silently confused in downstream tooling.

Risk L7 — Single period analysis on a volatile period. If the analysis period is an anomalous week (holiday spike, data pipeline outage), LiftMap will rank based on distorted rates. DataProfiler quality checks reduce but cannot eliminate this. Users are responsible for choosing a representative analysis period. The date coverage check and grain check help surface obvious data issues.

Risk L8 — Zero-denominator segments silently included in totals. Same risk as Movement Mode. Tagged "zero_denominator", excluded from totals, quality warning added. Test `test_lift_zero_denominator_segment` verifies this.

43. LiftMap Build Checklist

Core Scoring

- $\text{opportunity_signal} = D \times \max(\text{benchmark_rate} - r, 0)$ for every segment
- $\text{outperformance_signal} = D \times \max(r - \text{benchmark_rate}, 0)$ when `outperformance_mode=True`
- `benchmark_rate` defaults to population average $R = \sum N / \sum D$
- `benchmark_source` = "population_average" | "user_provided"
- `opportunity_rank` = 1 for highest signal, None for signal = 0
- `zero_denominator` segments excluded from totals and ranking

Validation

- `DecompositionTypeError` raised for `SumMetric` and `CountMetric` with correct message
- `ZeroDenominatorError` raised when overall denominator = 0
- `ValueError` raised for non-float `benchmark_rate`
- WARN when user benchmark outside observed segment rate range
- WARN when all segments above benchmark (`opportunity` = 0)

DataProfiler

- All four standard quality checks run on analysis period
- Benchmark coverage check (Check L1) implemented
- Quality check results appear in Section 2 of every report

Report

- All seven sections present in JSON, Markdown, HTML
- `interpretation_note` equals mandatory text exactly
- Opportunity ranking footnote present in Section 4
- Outperformance section present only when `outperformance_mode=True`
- `investigation_areas` header uses correct non-causal language

Schema

- `LiftResult` schema clean — no Movement Mode fields
- `LiftMetadataBlock` includes `benchmark_rate`, `benchmark_source`, `analysis_period`
- `schema_version` = "2.0"
- `QualityCheckResult` reused from Movement Mode schema

Tests

- `test_lift_opportunity_signal_formula` passes
- `test_lift_default_benchmark_is_population_average` passes
- `test_lift_all_above_benchmark`: zero signal, WARN, no error
- `test_lift_additive_raises`: `DecompositionTypeError`
- `test_lift_interpretation_note_present` in all three formats
- `test_lift_no_movement_mode_schema_fields`: clean schema separation
- All tests pass on Python 3.10, 3.11, 3.12

Package

- `lens.lift()` method on `MetricLens` object
- `LiftResult` not exported from `__init__.py` until v2 branch is released

- No LiftMap code, stubs, or TODOs in v0.1 or v1 codebase
- CHANGELOG.md has v2.0.0 entry when LiftMap ships

44. Master Roadmap Summary

Feature	v0.1.0	v1.0	v2.0
Movement Mode	YES	YES (expanded)	YES
SumMetric, CountMetric	YES	YES	YES
RatioMetric, AverageMetric	YES	YES	YES
Additive decomposition	YES	YES	YES
Three-way mix/rate/cross	YES	YES	YES
DataProfiler (4 checks)	YES	YES	YES
JSON/Markdown/HTML output	YES	YES	YES
pytest suite	YES	YES	YES
Synthetic demo	YES	YES	YES
CLI (metriclens rca)	no	YES	YES
Bootstrap confidence intervals	no	YES	YES
Shapley across dimensions	no	YES	YES
Multi-metric (analyze_many)	no	YES	YES
Optional LLM narrator	no	YES	YES
LiftMap Mode	no	no	YES
LiftMap opportunity ranking	no	no	YES
LiftMap outperformance mode	no	no	YES
Streamlit demo app	no	no	YES
Multi-dimension joint analysis	no	no	YES
dbt/MetricFlow integration	no	no	YES

MetricLens Master PRD — May 2026

Package: v0.1.0 (Movement Mode build-ready) · v1.0 (planned) · v2.0 (LiftMap, planned)

Schema versions: "0.1" (v0.1.0) · "1.0" (v1.0) · "2.0" (v2.0)

Solo-built · Open-source · Parked side project

Complements MetaSignal / RiskFrame / DevPulse / PulseRank without overlapping their use cases